AI / LLM / Agents / Hardware Notes

Foundations · Architecture · Systems · Frontiers

Jeriel Chan

March 18, 2026

Contents

1 Math & Probability Refresher **9**
1.1 Linear algebra essentials . . . 9
1.2 Probability & information theory . . . 9
1.3 Calculus & gradients . . . 9
1.4 Statistics for ML . . . 9

2 Classical ML & Optimization **9**
2.1 Supervised learning fundamentals . . . 9
2.2 Gradient descent variants . . . 9
2.3 Convex vs non-convex optimization . . . 9
2.4 SVMs, trees, ensembles . . . 9

3 Neural Networks from Scratch **9**
3.1 Perceptrons & MLPs . . . 9
3.2 Activation functions . . . 9
3.3 Initialization strategies . . . 9
3.4 Universal approximation theorem . . . 9

4 Backpropagation & Autodiff **9**
4.1 Computational graphs . . . 9
4.2 Forward & reverse mode autodiff . . . 9
4.3 Vanishing & exploding gradients . . . 9
4.4 PyTorch autograd internals . . . 9

5 Regularization, Loss Functions & Training Tricks **9**
5.1 L1, L2, dropout, batch norm . . . 9
5.2 Loss function zoo . . . 9
5.3 Learning rate schedules . . . 9
5.4 Mixed precision training . . . 9

6 Transformer Architecture Deep Dive **9**
6.1 Encoder-decoder vs decoder-only . . . 9
6.2 Multi-head self-attention . . . 9
6.3 Feed-forward sublayer . . . 9
6.4 Layer norm & residual stream . . . 9

7 Attention Mechanisms **9**
7.1 Scaled dot-product attention . . . 9
7.2 Multi-head attention (MHA) . . . 9

7.3 Multi-query & grouped-query attention (MQA, GQA) . . . 9
7.4 Sparse & linear attention variants . . . 9
7.5 FlashAttention . . . 9

8 Positional Encodings **9**
8.1 Sinusoidal encodings (original) . . . 9
8.2 Learned absolute positions . . . 9
8.3 Relative position encodings . . . 9
8.4 RoPE (rotary position embedding) . . . 9
8.5 ALiBi . . . 9

9 Tokenization & Vocabulary **9**
9.1 BPE, WordPiece, Unigram . . . 9
9.2 SentencePiece & tiktoken . . . 9
9.3 Vocabulary size trade-offs . . . 9
9.4 Multilingual tokenization . . . 9

10 Scaling Laws & Emergent Capabilities **9**
10.1 Chinchilla scaling laws . . . 9
10.2 Compute-optimal training . . . 9
10.3 Emergent abilities & phase transitions . . . 9
10.4 Scaling inference compute . . . 9

11 Pretraining Pipelines & Data Curation **9**
11.1 Web-scale data collection . . . 9
11.2 Deduplication & quality filtering . . . 9
11.3 Data mixing strategies . . . 9
11.4 Pretraining objectives (CLM, MLM, etc.) . . . 9

12 Fine-tuning: SFT, LoRA, QLoRA **9**
12.1 Supervised fine-tuning (SFT) . . . 9
12.2 Parameter-efficient fine-tuning (PEFT) . . . 9
12.3 LoRA & AdaLoRA . . . 9
12.4 QLoRA & quantized fine-tuning . . . 9

13 RLHF, DPO & Reward Modelling **9**
13.1 Human preference collection . . . 9
13.2 Reward model training . . . 9
13.3 PPO for language models . . . 9
13.4 Direct preference optimization (DPO) . . . 9
13.5 GRPO & alternatives . . . 9

14 Mixture of Experts (MoE) **9**
14.1 Sparse MoE fundamentals . . . 9
14.2 Routing mechanisms . . . 9
14.3 Load balancing losses . . . 9
14.4 MoE in practice (Mixtral, GPT-4 rumours) . . . 9

15 Inference & Decoding Strategies **9**
15.1 Greedy, beam search, sampling . . . 9
15.2 Temperature, top-k, top-p . . . 9
15.3 KV cache mechanics . . . 9
15.4 Speculative decoding . . . 9
15.5 Continuous batching . . . 9

16 Quantization & Model Compression **9**
16.1 Post-training quantization (PTQ) . . . 9
16.2 Quantization-aware training (QAT) . . . 9
16.3 GPTQ, AWQ, GGUF formats . . . 9
16.4 Pruning & distillation . . . 9

17 Prompt Engineering & In-context Learning **9**
17.1 Zero-shot & few-shot prompting . . . 9
17.2 Chain-of-thought (CoT) . . . 9
17.3 Structured output prompting . . . 9
17.4 System prompts & jailbreaks . . . 9

18 RAG & Long-context Retrieval **9**
18.1 Naive RAG pipeline . . . 9
18.2 Embedding models & vector DBs . . . 9
18.3 Chunking strategies . . . 9
18.4 Reranking & advanced RAG . . . 9
18.5 Long-context vs retrieval trade-offs . . . 9

19 AI Agent Fundamentals & Architectures **9**
19.1 Agent loop: perceive, plan, act . . . 9
19.2 ReAct framework . . . 9
19.3 Reflexion & self-critique . . . 9
19.4 Agent scaffolding patterns . . . 9

20 Tool Use, Function Calling & MCP **9**
20.1 Function calling APIs . . . 9
20.2 Model Context Protocol (MCP) . . . 9
20.3 Tool schema design . . . 9
20.4 Error handling in tool loops . . . 9

21 Multi-agent Systems & Orchestration **9**
21.1 Orchestrator-subagent patterns . . . 9
21.2 AutoGen, CrewAI, LangGraph . . . 9
21.3 Inter-agent communication . . . 9
21.4 Swarm intelligence analogies . . . 9

22 Agent Memory: Episodic, Semantic, Working **9**
22.1 In-context (working) memory . . . 9
22.2 External episodic memory . . . 9
22.3 Semantic memory via vector DBs . . . 9
22.4 Memory consolidation strategies . . . 9

23 Planning: MCTS, CoT, o1-style Reasoning **9**
23.1 Classical planning (STRIPS, PDDL) . . . 9
23.2 Monte Carlo Tree Search (MCTS) . . . 9
23.3 Chain-of-thought & scratchpads . . . 9
23.4 o1 / o3 test-time compute scaling . . . 9

24 World Model Theory **9**
24.1 LeCun's JEPA architecture . . . 9
24.2 Dreamer (RSSM) . . . 9
24.3 World models in RL . . . 9
24.4 Predictive coding theories . . . 9

25 Video & Predictive Models 9
25.1 Sora & video diffusion . . . 9
25.2 Genie & interactive world models . . . 9
25.3 Temporal consistency challenges . . . 9
25.4 Next-frame prediction as pretraining . . . 9

26 Embodied AI & Simulation 9
26.1 Sim-to-real transfer . . . 9
26.2 RT-2, PaLM-E, embodied LLMs . . . 9
26.3 Simulation environments (Isaac, MuJoCo) . . . 9
26.4 Manipulation & locomotion tasks . . . 9

27 Vision-Language Models 9
27.1 CLIP & contrastive pretraining . . . 9
27.2 LLaVA, Flamingo, PaLI . . . 9
27.3 Vision tokens & patch embeddings . . . 9
27.4 Grounding & spatial reasoning . . . 9

28 Diffusion Models & Generative Vision 9
28.1 DDPM & DDIM . . . 9
28.2 Score matching & SDE perspective . . . 9
28.3 Stable Diffusion architecture . . . 9
28.4 ControlNet, LoRA for images . . . 9
28.5 Flow matching . . . 9

29 Audio, Speech & Multimodal Fusion 9
29.1 Whisper & speech recognition . . . 9
29.2 Audio tokenization . . . 9
29.3 TTS & voice cloning . . . 9
29.4 Multimodal fusion strategies . . . 9

30 GPU Architecture 9
30.1 SM, warp, thread hierarchy . . . 9
30.2 Tensor cores & TF32/BF16 . . . 9
30.3 Memory: HBM, L2, SRAM . . . 9
30.4 CUDA programming model . . . 9
30.5 NVLink & GPU interconnects . . . 9

31 TPUs, NPUs & Custom ASICs 9
31.1 Google TPU architecture . . . 9
31.2 Systolic arrays . . . 9
31.3 Apple Neural Engine . . . 9
31.4 Groq LPU & Cerebras WSE . . . 9
31.5 Custom ASIC design trade-offs . . . 9

32 Memory Hierarchy & Bandwidth Bottlenecks 9
32.1 Roofline model . . . 9
32.2 Arithmetic intensity . . . 9
32.3 Memory-bound vs compute-bound ops . . . 9
32.4 HBM bandwidth & capacity limits . . . 9

33 Distributed Training 9
33.1 Data parallelism (DDP) . . . 9
33.2 Tensor parallelism . . . 9
33.3 Pipeline parallelism . . . 9

33.4 FSDP & ZeRO optimizer . . . 9
33.5 3D parallelism . . . 9

34 Inference Optimization on Hardware **9**
34.1 FlashAttention 1, 2, 3 . . . 9
34.2 Paged KV cache (vLLM) . . . 9
34.3 Continuous batching engines . . . 9
34.4 TensorRT & compilation . . . 9

35 Running LLMs On-device **9**
35.1 llama.cpp & GGUF . . . 9
35.2 Ollama & LM Studio . . . 9
35.3 MLX (Apple Silicon) . . . 9
35.4 Quantization for edge . . . 9
35.5 Privacy & latency trade-offs . . . 9

36 OpenClaw & Agent OS Integration **9**
36.1 OpenClaw architecture overview . . . 9
36.2 OS-level agent primitives . . . 9
36.3 Filesystem & process access . . . 9
36.4 Security sandboxing . . . 9
36.5 Desktop automation patterns . . . 9

37 Embedded AI & Microcontroller Inference **9**
37.1 TensorFlow Lite & TFLite Micro . . . 9
37.2 ONNX Runtime on ARM . . . 9
37.3 Keyword spotting pipelines . . . 9
37.4 Power & memory constraints . . . 9

38 Edge Deployment **9**
38.1 Model pruning for edge . . . 9
38.2 Post-training quantization (INT4, INT8) . . . 9
38.3 ONNX export & optimization . . . 9
38.4 Neural Architecture Search (NAS) . . . 9

39 CUDA Kernels for ML **9**
39.1 Matrix multiplication (GEMM) . . . 9
39.2 Softmax & layer norm kernels . . . 9
39.3 Fused attention kernel . . . 9
39.4 Shared memory tiling . . . 9
39.5 Profiling with Nsight . . . 9

40 Triton & Custom Op Authoring **9**
40.1 Triton language overview . . . 9
40.2 Writing a fused kernel in Triton . . . 9
40.3 Triton vs CUDA trade-offs . . . 9
40.4 Integration with PyTorch . . . 9

41 Profiling & Optimization Workflows **9**
41.1 PyTorch Profiler & trace viewer . . . 9
41.2 Nsight Systems & Compute . . . 9
41.3 Bottleneck identification workflow . . . 9
41.4 Operator fusion strategies . . . 9

42 AI Safety Landscape & Threat Models **9**

42.1 Alignment problem overview . . . 9
42.2 Reward hacking & Goodhart's law . . . 9
42.3 Deceptive alignment . . . 9
42.4 Containment & control . . . 9

43 Constitutional AI & RLHF Safety **9**
43.1 Constitutional AI (Anthropic) . . . 9
43.2 RLHF safety properties . . . 9
43.3 Red-teaming methodologies . . . 9
43.4 Refusal & harmlessness trade-offs . . . 9

44 Interpretability & Mechanistic Analysis **9**
44.1 Probing classifiers . . . 9
44.2 Superposition hypothesis . . . 9
44.3 Sparse autoencoders (SAE) . . . 9
44.4 Circuit-level analysis . . . 9
44.5 Activation patching . . . 9

45 Evaluation Methodology & Benchmark Design **9**
45.1 Evaluation taxonomy . . . 9
45.2 Contamination & data leakage . . . 9
45.3 Human vs automated evaluation . . . 9
45.4 Statistical significance in evals . . . 9

46 Standard Benchmarks **9**
46.1 MMLU, HellaSwag, ARC . . . 9
46.2 HumanEval, SWE-bench . . . 9
46.3 MT-Bench & Chatbot Arena . . . 9
46.4 MATH, GSM8K . . . 9
46.5 GPQA & frontier benchmarks . . . 9

47 Evals for Agents & Agentic Tasks **9**
47.1 WebArena, OSWorld . . . 9
47.2 SWE-bench Verified . . . 9
47.3 Tool-use evals . . . 9
47.4 Long-horizon task evaluation . . . 9
47.5 Safety evals for agents . . . 9

48 State Space Models: Mamba & RWKV **9**
48.1 S4 & structured state spaces . . . 9
48.2 Mamba & selective SSMs . . . 9
48.3 RWKV: RNN-transformer hybrid . . . 9
48.4 SSM vs attention trade-offs . . . 9

49 Test-time Compute & Reasoning Models **9**
49.1 Scaling inference compute . . . 9
49.2 OpenAI o1 / o3 architecture (known) . . . 9
49.3 Process reward models (PRMs) . . . 9
49.4 Best-of-N & self-consistency . . . 9
49.5 Thinking tokens & scratchpads . . . 9

50 Neurosymbolic AI & Program Synthesis **9**
50.1 Neural-symbolic integration . . . 9
50.2 Program induction . . . 9
50.3 AlphaCode & code generation . . . 9

50.4 Formal verification + LLMs . . . 9

51 Research Reading Log & Paper Notes **9**
51.1 How to use this chapter . . . 9
51.2 Paper template . . . 9
51.3 Reading list . . . 9

52 AI Coding Agents & IDEs **9**
52.1 Landscape overview . . . 9
52.2 Cursor: architecture & edit model . . . 9
52.3 GitHub Copilot & Codex . . . 9
52.4 Windsurf (Codeium) . . . 9
52.5 Antigravity . . . 9
52.6 Devin & SWE-agent . . . 9
52.7 Aider & CLI coding agents . . . 9
52.8 How AI IDEs use context (repo map, RAG) . . . 9
52.9 Diff application & conflict resolution . . . 9
52.10 Evaluation: SWE-bench, IDE evals . . . 9

53 Agentic Platforms: Manus, Computer Use & Beyond **9**
53.1 Manus AI overview (Butterfly Effect) . . . 9
53.2 Anthropic Computer Use . . . 9
53.3 OpenAI Operator & Agents SDK . . . 9
53.4 Browser-use & web automation . . . 9
53.5 Desktop agent architectures . . . 9
53.6 Security & sandboxing for autonomous agents . . . 9
53.7 Meta's agent research (LLaMA, CICERO) . . . 9

54 University AI Modules **9**
54.1 CS2109S — AI and Machine Learning . . . 9
54.2 CS3263 — Foundations of AI . . . 10
54.3 CS4246 — AI Planning and Decision Making . . . 10
54.4 MA4275 — Mathematics for Reinforcement Learning . . . 10

55 Distillation, Quantization, and LLM/Agent Glossary **11**
55.1 Distillation: Pros and Cons . . . 11
55.2 Quantization . . . 11
55.3 Distillation vs Quantization . . . 12
55.4 Fast Refresher: Common LLM Terms . . . 12
55.5 Fast Refresher: Common Agent Terms . . . 13
55.6 Highest-yield Terms (Quick Memory) . . . 14
55.7 One Clean Mental Model . . . 14

Math & Probability Refresher

Linear algebra essentials

Probability & information theory

Calculus & gradients

Statistics for ML

Classical ML & Optimization

Supervised learning fundamentals

Gradient descent variants

Convex vs non-convex optimization

SVMs, trees, ensembles

Neural Networks from Scratch

Perceptrons & MLPs

Activation functions

Initialization strategies

Universal approximation theorem

Backpropagation & Autodiff

Computational graphs

Forward & reverse mode autodiff

Vanishing & exploding gradients

PyTorch autograd internals

Regularization, Loss Functions & Training Tricks

L1, L2, dropout, batch norm

Loss function zoo

Learning rate schedules

Mixed precision training

Transformer Architecture Deep Dive

Encoder-decoder vs decoder-only

Multi-head self-attention

Feed-forward sublayer

Layer norm & residual stream

Attention Mechanisms

Scaled dot-product attention

Multi-head attention (MHA)

Multi-query & grouped-query attention (MQA, GQA)

CS3263 — Foundations of AI

Search algorithms, knowledge representation, and reasoning.

CS4246 — AI Planning and Decision Making

Automated planning, reinforcement learning, decision-theoretic approaches.

MA4275 — Mathematics for Reinforcement Learning

Mathematical foundations: dynamic programming, Markov decision processes, value functions.

Distillation, Quantization, and LLM/Agent Glossary

Distillation: Pros and Cons

Distillation is training a smaller *student* model to imitate a stronger *teacher* model.

Pros

- **Cheaper model, decent performance**: A much smaller model can behave surprisingly well by learning from a stronger one.
- **Faster inference**: Smaller models usually respond faster and cost less to run.
- **Task specialization**: You can distill a model to be especially good at coding, math, support replies, and similar tasks.
- **Cleaner training signal**: Teacher outputs are often more structured and polished than raw internet data.
- **Good for local / edge deployment**: Distilled models are often the ones that fit on consumer GPUs or laptops.

Cons

- **Student inherits teacher mistakes**: If the teacher hallucinates or has odd style habits, the student can learn that too.
- **Can look smarter than it is**: The student may imitate reasoning patterns without improving deep capability.
- **Benchmark overfitting risk**: If synthetic data resembles eval data, results can look inflated.
- **Ceiling is limited**: The student usually will not robustly surpass the teacher without additional factors.
- **Legal / ethical gray area**: Using proprietary model outputs for training can be controversial depending on terms and jurisdiction.
- **Mode collapse / sameness**: Overtraining on teacher outputs can make the student narrow, repetitive, or less creative.

Quantization

Quantization means using lower-precision numbers so models use less memory and run faster.

LLMs store many weights. Common numeric formats:

- **FP32**: 32-bit floating point
- **FP16 / BF16**: 16-bit formats
- **INT8 / INT4 / 4-bit**: smaller integer-like representations

Instead of storing an exact value like `0.18372642`, you store a compressed representation (for example, a bucketed value on a scale). It is not exact, but often close enough.

Why quantize?

- Less VRAM / RAM usage
- Faster inference
- Lets bigger models fit on smaller hardware

- Very useful for local LLMs

Trade-off

- More compression usually means more accuracy loss.
- Some quantized models can become worse at reasoning, coding, long-context consistency, and rare edge cases.

Intuition

Full precision is like a high-resolution image. Quantization is like compression: often visually similar, but fine details may degrade.

Common quantization levels

- **8-bit**: usually pretty safe
- **4-bit**: very common for local inference
- **2-bit / ultra-low-bit**: aggressive, can hurt quality significantly

Distillation vs Quantization

- **Distillation**: train a smaller model to imitate a bigger one.
- **Quantization**: compress how a model's numbers are stored and run.

So:

- Distillation changes what the model learned and often its effective size/capability profile.
- Quantization changes storage and inference representation.

Fast Refresher: Common LLM Terms

Pretraining
Training on huge raw datasets (text, code, books, web pages). This is where the base model learns language/world patterns.

Fine-tuning
Training a pretrained model on a narrower dataset to specialize it.

SFT (Supervised Fine-Tuning)
Fine-tuning with input-output examples.

Synthetic data
Training data generated by another model rather than humans.

RLHF
Reinforcement Learning from Human Feedback: humans rate outputs, then the model is optimized for preferred behavior.

RLAIF
Similar to RLHF, but feedback comes from AI systems instead of humans.

Alignment
Making model behavior helpful, safe, and consistent with intended values.

Inference
Running the model to generate outputs.

Parameters
Learned weights in a model (for example, "7B" means about 7 billion parameters).

Context window
How much text the model can process at once.

Embeddings
Vector representations used for similarity search, retrieval, and clustering.

RAG
Retrieval-Augmented Generation: fetch relevant documents first, then answer using them.

Hallucination
Confidently generating false or unsupported information.

Temperature
Randomness control: lower is more deterministic; higher is more creative/chaotic.

Top-k / Top-p
Sampling controls that shape generation randomness.

MoE (Mixture of Experts)
Architecture where only some expert subnetworks activate per token.

LoRA
Lightweight fine-tuning by training low-rank adapters instead of all weights.

Adapter
Small modules attached to a base model for specialization.

Checkpoint
A saved model state.

Benchmark
A standard test suite for comparing models.

Contamination
Benchmark questions (or close variants) leaking into training data.

Fast Refresher: Common Agent Terms

Agent
A system where the model does more than chat: plans, uses tools, calls APIs, reads files, and may loop until a goal is complete.

Tool use
Calling external functions such as web search, code execution, or database queries.

Function calling
Structured tool invocation interface used by a model.

Planner
Component that decides steps.

Executor
Component that performs steps.

ReAct
Pattern that alternates reasoning and acting.

Memory
Stored information from prior steps or sessions.

Short-term memory
Current-task scratchpad/context.

Long-term memory
Persistent saved preferences/history/knowledge.

Reflection
Agent critiques its own output and revises.

Multi-agent
Multiple agents with distinct roles collaborating.

Orchestrator
Controller coordinating tools, agents, and workflow.

MCP
A standard protocol/interface pattern that connects models to tools and context.

Workflow vs agent
Workflow is fixed-step execution; agent behavior is dynamic decision-making.

Highest-yield Terms (Quick Memory)

- **Pretraining** = learn from large raw corpora
- **Fine-tuning** = specialize the model
- **Distillation** = learn from another model
- **Quantization** = compress numerical representation
- **RAG** = retrieve documents before answering
- **Agent** = plan and use tools dynamically

One Clean Mental Model

Think about building an AI system this way:

- Pretraining gives the brain
- Fine-tuning gives the job
- Distillation makes it cheaper
- Quantization makes it lighter
- RAG gives references
- Agents/tools let it do things